

PSEUDOCODE WRITING TASK

Questions 31, 32 and 33

Name: N. PARNITA

Register No.: 192525322

Course code: CSA0617

Course Name: Design and Analysis of Algorithm

Q31. N-Queens Problem with Pre-Placed Queen

Problem Definition

Given an $N \times N$ chessboard with one queen already fixed at a specified row and column, place the remaining $N-1$ queens so that no two queens attack each other. The pre-placed queen must remain unchanged.

Inputs and Output

Input: Board size N , fixed row and fixed column. Output: A valid arrangement of N queens, or no solution if the fixed position makes the problem impossible.

Approach

The board is filled row by row using Backtracking. The fixed queen is placed first and is never changed. For every other row, each column is tested. A position is accepted only when its column and both diagonals are safe. If no position is possible, the algorithm backtracks to the previous row.

Pseudocode

```
N_QUEENS_WITH_FIXED(N, fixedRow, fixedCol)
    create board[N][N] and fill with EMPTY

    board[fixedRow][fixedCol] ← QUEEN

    if not SAFE_FIXED_QUEEN(board, fixedRow, fixedCol)
        return "No solution"

    if SOLVE(0)
        return board
    else
        return "No solution"

SOLVE(row)
    if row = N
        return TRUE

    if row = fixedRow
        return SOLVE(row + 1)

    for col ← 0 to N-1
        if IS_SAFE(board, row, col)
```

```

        board[row][col] ← QUEEN

        if SOLVE(row + 1)
            return TRUE

        board[row][col] ← EMPTY

    return FALSE

IS_SAFE(board, row, col)
    check same column
    check upper-left and lower-right diagonals
    check upper-right and lower-left diagonals

    if no queen attacks (row, col)
        return TRUE
    return FALSE

```

Result

The algorithm keeps the pre-placed queen fixed and searches for valid positions for all remaining queens. Invalid placements are rejected immediately, which prunes the search. If a complete arrangement is found, it is a valid N-Queens solution; otherwise, no solution exists for the given fixed position.

Q32. N-Queens Problem for Counting All Solutions

Problem Definition

The objective is to count the total number of valid ways to place N queens on an N×N chessboard such that no two queens share the same row, column, or diagonal.

Inputs and Output

Input: Board size N. Output: Total number of valid N-Queens arrangements.

Approach

Backtracking places one queen in each row. Every safe column is tried, and after recursively solving the next row, the queen is removed so that another configuration can be explored. Unlike the normal N-Queens search, the algorithm does not stop after finding one solution; it counts every valid arrangement.

Pseudocode

```

COUNT_N_QUEENS(N)
    create board[N][N] and fill with EMPTY
    count ← COUNT_SOLUTIONS(0)
    return count

COUNT_SOLUTIONS(row)
    if row = N
        return 1

    total ← 0

```

```

    for col ← 0 to N-1
        if IS_SAFE(board, row, col)
            board[row][col] ← QUEEN

            total ← total + COUNT_SOLUTIONS(row + 1)

            board[row][col] ← EMPTY

    return total

IS_SAFE(board, row, col)
    check same column
    check both diagonals

    if no queen attacks (row, col)
        return TRUE
    return FALSE

```

Example

For $N = 4$, the algorithm counts 2 valid arrangements.

Result

The algorithm explores all valid queen placements and returns the total number of solutions without omitting any configuration. Safety checks prune invalid branches as soon as a queen conflicts with an existing queen.

Q33. Sudoku Solver with Fixed Cells Constraint

Problem Definition

Solve a partially filled 9×9 Sudoku grid while ensuring that all originally filled cells remain unchanged. Every row, column, and 3×3 subgrid must contain the numbers 1 to 9 without repetition.

Inputs and Output

Input: A partially filled 9×9 Sudoku grid. Output: A completed valid Sudoku grid, or failure if no solution exists.

Approach

The algorithm finds an empty cell and tries numbers from 1 to 9. A number is placed only when it does not already occur in the same row, column, or 3×3 subgrid. Original cells are skipped and can never be modified. If a later placement causes failure, the algorithm backtracks and tries another number.

Pseudocode

```

SOLVE_SUDOKU(grid)
    find the next empty cell (row, col)

    if no empty cell exists
        return TRUE

```

```

    for num ← 1 to 9
        if IS_VALID(grid, row, col, num)
            grid[row][col] ← num

            if SOLVE_SUDOKU(grid)
                return TRUE

            grid[row][col] ← EMPTY

    return FALSE

IS_VALID(grid, row, col, num)
    if num already exists in row
        return FALSE

    if num already exists in column
        return FALSE

    if num already exists in the 3×3 subgrid
        return FALSE

    return TRUE

```

Fixed Cells Constraint

The initially filled cells are treated as fixed values. The solver searches only for empty cells, so an original value is never overwritten or changed.

Result

The backtracking algorithm produces a complete valid Sudoku grid when a solution exists. Invalid numbers are rejected immediately using row, column, and subgrid checks, which reduces unnecessary search.

Conclusion

Q31 uses Backtracking to complete an N-Queens board while preserving a fixed queen. Q32 modifies the same technique to count every valid N-Queens arrangement. Q33 applies Backtracking to Sudoku while preserving all pre-filled cells and checking the three Sudoku constraints.